DIGT2107: Practice of Software Development

JWTOffline

Iteration 3 (Deliverable 2.1): Testing and Initial Development

Course: DIGT2107, Fall Term 2026

Instructor: Dr. May Haidar

Team Number: [Team #]

Team Members: Isa Din, Steven Passynkov, Osmond Jiang, Saeed Rafieepour

Submitted: November 8, 2026

Repository: https://github.com/isad07/JWToffline (tag ITR2.1)

Contents

1. Introduction

2. Iteration Goals and Objectives

3. Test Plan

3.1 Scope, 3.2 Test Levels and Types, 3.3 Tools and Environment

3.4 Entry and Exit Criteria, 3.5 How to Run the Tests, 3.6 Defect Handling

4. Requirement to Test Case Traceability

4.1 Functional, 4.2 Non-Functional, 4.3 Validation, 4.4 Gaps Found

5. Detailed Test Cases (TC-001 to TC-058)

6. Test Results

6.1 Summary, 6.2 Automated Test Inventory, 6.3 Defect Log

7. Prototype Overview

7.1 Changes Since ITR1.2, 7.2 Requirement Updates, 7.3 Running the Prototype

8. Code Repository and Branching Strategy

9. Task Allocation and Timeline

10. Updated Iteration Plan

1. Introduction

Project Name: JWTOffline

Team Number: [Team #]

Team Members: Isa Din, Steven Passynkov, Osmond Jiang, Saeed Rafieepour

JWTOffline is a desktop app for decoding, building, and checking JSON Web Tokens without pasting them into a website. In Deliverable 1.2 we wrote 23 functional and 8 non-functional requirements and built a prototype of all five screens. This iteration is about proving those requirements hold up.

We did three things. First, we wrote a test plan and a traceability matrix that ties every requirement to at least one test case. Second, we filled the gaps the matrix showed: the workspace now saves to a file (FR-22), and we added performance, robustness, and code structure tests for the non-functional requirements. Third, writing those tests turned up a real crash, which we fixed and logged as defect D-01.

The suite now has **90 JUnit tests** in 12 test classes, grouped into **58 documented test cases** (47 automated, 11 manual).

How this document is organized

- **Section 2** lists the goals for this iteration and what we delivered.
- **Section 3** is the test plan: what we test, how, with which tools, and when a test counts as passed.
- **Section 4** is the requirement to test case traceability matrix with coverage status and known gaps.
- **Section 5** has every test case in detail.
- **Section 6** summarizes results so far and the defect log.
- **Section 7** covers the prototype changes and the requirement updates since Deliverable 1.2.
- **Sections 8 to 10** cover the repository, task allocation, the sprint plan, and the updated iteration plan.

2. Iteration Goals and Objectives

Goal from the spec	What we delivered
Implement core functionality linked to high priority user stories	All high priority stories were already working in ITR1.2. This iteration finished the last open story in the backlog, US-4.5: the workspace is now stored in ~/.jwtoffline/workspace.json and reloaded on startup.
Develop and run unit tests mapped to specific requirements	20 new tests (90 total). Every test class names the requirements and test cases it covers at the top of the file, and every test method is listed in a test case in Section 5.
Deliver a prototype that shows how tests relate to requirements	Traceability matrix in Section 4 links all 31 requirements to test cases and test methods. The same matrix is committed as a CSV in docs/deliverable-2.1.

3. Test Plan

3.1 Scope

In scope: all 23 functional requirements and all 8 non-functional requirements from Deliverable 1.2, as updated in Section 7.2.

Out of scope for now: automated GUI tests of the JavaFX screens. The Deliverable 3.1 spec asks for GUI testing with a coverage report, so we will add TestFX tests then. Until that point the screens are covered by the manual test cases in this document.

3.2 Test Levels and Types

Level / type	What it checks	How	Test cases
Unit	One class at a time: decoder, encoder, verifier, claim inspector, JSON, key parser, repositories, history.	JUnit 5, fixed clocks and fixed keys so results are repeatable.	TC-001 to TC-024, TC-026 to TC-032, TC-034 to TC-036, TC-039 to TC-042, TC-046
Integration	Several classes working together, for example saved sample tokens going through the decoder and verifier.	JUnit 5 using real objects, no mocks.	TC-037, TC-020
Performance	NFR-2 timing targets.	JUnit with warm up runs and the median of 21 timed runs.	TC-048, TC-049, TC-050
Robustness	NFR-5: nothing typed can crash the app.	Seeded random input (fuzzing) plus regression tests for each crash we find.	TC-042, TC-052, TC-053
Static checks	NFR-1 no network code, NFR-7 no UI code in logic packages.	JUnit tests that scan the source tree.	TC-045, TC-056
Coverage	NFR-7 line coverage target.	JaCoCo report from mvn verify.	TC-057
Manual UI and system	Screens, navigation, clipboard, restart, platforms, accessibility.	Written steps run by a team member, result recorded in this document.	TC-010, TC-025, TC-033, TC-038, TC-043, TC-044, TC-047, TC-050, TC-055, TC-058
Usability	NFR-3 first use.	Hallway test with three classmates.	TC-051

3.3 Tools and Environment

- **JUnit 5.10** for all automated tests. **JaCoCo 0.8.12** for coverage. **Maven** runs both with `mvn verify`.
- **GitHub Actions** runs `mvn -B verify` on Ubuntu with Temurin JDK 17 on every push to main, development, and feature branches, and on every pull request. The JaCoCo report is attached to each run.
- Developer machines: Windows 11 and macOS with JDK 17 or newer.

- Test data: the jwt.io example token, the three sample tokens in SampleTokens (secret `jwtoffline-demo-secret`), and RSA keys generated fresh in each test run. No real tokens or keys are used.

3.4 Entry and Exit Criteria

A feature is ready for testing when it compiles, its test cases are written in this document, and the code is on a feature branch.

A test case passes when every expected result in its row is met. For automated cases that means every assertion passes in CI. For manual cases the tester records what they saw in Actual Result.

A feature branch can merge into development when CI is green and a second team member has reviewed the pull request.

The iteration is done when all automated tests pass, every requirement has at least one test case, and every failure is in the defect log with a fix or a plan.

3.5 How to Run the Tests

Command	What it does
`mvn test`	Runs all 90 JUnit tests.
`mvn verify`	Runs the tests and writes the coverage report to target/site/jacoco/index.html.
`mvn -Dtest=PerformanceTest test`	Runs only one test class (here the NFR-2 timing tests).
`mvn javafx:run`	Starts the app for the manual test cases.

3.6 Defect Handling

A failing test or manual check gets an ID (D-01, D-02, ...) in Section 6.3 and a GitHub issue. The fix goes on a feature branch together with a regression test that failed before the fix. The defect is closed when that test passes in CI.

4. Requirement to Test Case Traceability

Each row links a requirement to the test cases that validate it. Test case details, including the exact JUnit methods, are in Section 5. Coverage status is our judgement of whether the listed test cases check every part of the requirement, which is a separate question from whether they have been run yet (see Section 6).

4.1 Functional Requirements

Req ID	Requirement	Test Case IDs	Test Case Description	Coverage
FR-1	Accept pasted token, ignore whitespace and "Bearer " prefix	TC-002	Accept a "Bearer " prefix and extra whitespace	Fully covered
FR-2	Split token and Base64URL decode header and payload locally	TC-001	Decode a valid HS256 token	Fully covered
FR-3	Show header and payload as indented JSON in original key order	TC-001, TC-003, TC-008	Decode a valid HS256 token; Show header and payload as indented JSON in original key order; Claim values are shown exactly as sent	Fully covered
FR-4	Plain language error, never crash, on malformed tokens	TC-004, TC-005, TC-006, TC-007, TC-016, TC-031, TC-052, TC-053	Empty input gives a plain message; Wrong number of parts gives a plain message; Bad Base64URL in the header is reported; Payload that is not JSON is reported; Bad time claims are flagged, not crashed on; Missing key and broken token are explained; Random input never crashes the logic; Number with a huge exponent (regression for D-01)	Fully covered
FR-5	Show exp, iat, nbf as readable local date and time	TC-015, TC-016	exp, iat, and nbf shown as readable local times; Bad time claims are flagged, not crashed on	Fully covered
FR-6	Status banner: valid, expired, not yet valid, no expiry, with time left or since	TC-011, TC-012, TC-013, TC-014, TC-017	Token with a future exp shows as valid; Token past exp shows as expired; Token before nbf shows as not valid yet; Token without exp shows no expiry; Durations read naturally	Fully covered
FR-7	Show signing algorithm and signature length	TC-009	Signing algorithm is read from the header	Partially covered
FR-8	One click copy of header, payload, generated token	TC-010	Copy header, payload, and generated token	Fully covered (manual)
FR-9	Validate typed JSON before signing and point to the error position	TC-021	Invalid JSON is caught before signing	Partially covered

Req ID	Requirement	Test Case IDs	Test Case Description	Coverage
FR-10	Support HS256/384/512 and RS256/384/512, set alg and typ automatically	TC-019, TC-020	alg and typ are set from the chosen algorithm; All six algorithms sign and verify	Fully covered
FR-11	Sign with an HMAC secret or RSA private key (PKCS#8 or PKCS#1 PEM)	TC-018, TC-020, TC-022, TC-023, TC-024	Encoder reproduces a known HS256 token; All six algorithms sign and verify; Missing or unusable keys are explained; PKCS#1 RSA private keys are accepted; Numbers in claims are not rounded when signing	Fully covered
FR-12	Open a newly generated token in the decoder	TC-025	Open a generated token in the decoder	Fully covered (manual)
FR-13	Verify HMAC signed tokens against a secret	TC-026, TC-027, TC-028	Correct HMAC secret passes; Wrong HMAC secret fails with a reason; Edited payload is detected	Fully covered
FR-14	Verify RSA signed tokens against a PEM public key	TC-020, TC-029	All six algorithms sign and verify; RSA token checked against a public key	Fully covered
FR-15	Report pass or fail with a reason; refuse alg none and unsupported algorithms	TC-027, TC-030, TC-031, TC-032	Wrong HMAC secret fails with a reason; alg "none" and unsupported algorithms are refused; Missing key and broken token are explained; Signature check is separate from expiry	Fully covered
FR-16	Send the decoder token straight to the Verify screen	TC-033	Send the decoded token to the Verify screen	Fully covered (manual)
FR-17	Save a decoded token to the workspace with a label	TC-034, TC-037	Save a token with a label; Sample tokens decode and verify	Fully covered
FR-18	List saved tokens with algorithm, status, save time; filter by label	TC-035, TC-038	Find saved tokens by label, newest first; Workspace table and opening a saved token	Fully covered
FR-19	Rename or delete a saved token	TC-036	Rename and delete a saved token	Fully covered
FR-20	Open a saved token in the decoder	TC-038	Workspace table and opening a saved token	Fully covered (manual)
FR-21	Keep the 20 most recent tokens, newest first, no duplicates, on Home	TC-039	Recent history on the Home screen	Fully covered
FR-22	Store the workspace in a local file that survives closing the app	TC-040, TC-041, TC-042, TC-043	Workspace survives closing and reopening; First launch creates the workspace file; Damaged workspace file does not stop the app; Saved tokens are there after restarting the app	Fully covered

Req ID	Requirement	Test Case IDs	Test Case Description	Coverage
FR-23	Single window with side navigation that keeps each screen's input	TC-044	Side navigation keeps each screen's input	Fully covered (manual)

4.2 Non-Functional Requirements

The spec does not ask for a coverage status on non-functional requirements, so this table lists the test cases only.

Req ID	Requirement	Test Case IDs	Test Case Description
NFR-1	Privacy: zero network calls, secrets never written to disk	TC-045, TC-046, TC-047	No networking code in the app; Secrets are never written to disk; App works with the network turned off
NFR-2	Performance: 8 KB token decoded or verified in under 100 ms, startup under 3 s	TC-048, TC-049, TC-050	Decode an 8 KB token in under 100 ms; Verify an 8 KB token in under 100 ms; App starts in under 3 seconds
NFR-3	Usability: first token decoded within 30 s without instructions	TC-051	New user decodes a token within 30 seconds
NFR-4	Security: constant time comparison, alg none never verified, chosen algorithm decides signing	TC-019, TC-028, TC-030	alg and typ are set from the chosen algorithm; Edited payload is detected; alg "none" and unsupported algorithms are refused
NFR-5	Reliability: no input can crash the app	TC-042, TC-052, TC-053	Damaged workspace file does not stop the app; Random input never crashes the logic; Number with a huge exponent (regression for D-01)
NFR-6	Portability: Windows, macOS, Linux with JDK 17+, pure Java libraries only	TC-054, TC-055	Build and tests pass on Linux with JDK 17; App runs on Windows and macOS
NFR-7	Maintainability: logic separate from UI, storage behind an interface, 80% line coverage	TC-056, TC-057	Logic packages have no UI code; Line coverage of core and workspace is at least 80%
NFR-8	Accessibility: keyboard reachable, WCAG AA contrast, status in words	TC-058	Keyboard only use and readable status

4.3 How the Requirements Are Validated

- **Every one of the 31 requirements has at least one test case.** 18 of the 23 functional requirements are checked by automated tests. The other 5 (FR-8, FR-12, FR-16, FR-20, FR-23) are about what a screen shows or where a button leads, so they are manual for now, as are the table columns in FR-18.
- **Most requirements are covered from more than one side.** For example FR-4 (never crash on a bad token) is checked by the decoder error tests (TC-004 to TC-007), by the time claim tests (TC-016), and by the robustness tests that throw thousands of random inputs at it (TC-052, TC-053).

- **Non-functional requirements are tested too, not just reviewed.** NFR-1 and NFR-7 have tests that scan the source code, NFR-2 has timing tests, and NFR-5 has fuzz tests. Only NFR-3, NFR-6 on Windows and macOS, and NFR-8 need people.

4.4 Gaps Found

Gap	Requirement	Detail	Plan
G-1	FR-7	The algorithm name is tested, but the signature length shown on the Decode screen is only checked by eye.	Add a signatureLength() helper to DecodedJwt with a unit test (Iteration 2.2).
G-2	FR-9	TC-021 checks that invalid JSON is rejected but not that the message gives the position.	Assert "line 1 column" in the message (Iteration 2.2).
G-3	FR-8, FR-12, FR-16, FR-18, FR-20, FR-23	Screen behaviour has manual tests only.	TestFX GUI tests in Deliverable 3.1.
G-4	FR-22	If the workspace file cannot be read, the repository has a warning message, but the Workspace screen does not show it yet.	Show loadWarning() in a banner on the Workspace screen (Iteration 2.2).
G-5	NFR-2	Startup time (under 3 s) is a stopwatch check.	Keep manual. Run on two machines in TC-050.

5. Detailed Test Cases

Each test case follows the format from the course spec. **Automated by** names the JUnit test methods as Class#method so a marker can open the file and find them. Status is Pass, Fail, Pending (automated, written this iteration, waiting for its first CI run), or Not run (manual, scheduled).

5.1 Decode (FR-1 to FR-4, FR-7, FR-8)

Test Case ID	TC-001
Title	Decode a valid HS256 token
Requirement(s)	FR-2, FR-3
Type	Unit
Automated by	`JwtDecoderTest#decodesHeaderAndPayload`
Preconditions	The well known jwt.io example token is available (HS256, sub 1234567890, name John Doe).
Test Steps	1. Call JwtDecoder.decode() with the example token. 2. Read the header map, payload map, and signature segment.
Expected Result	Header has alg HS256 and typ JWT. Payload has sub "1234567890", name "John Doe", iat 1516239022. Signature segment is returned unchanged.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-002
Title	Accept a "Bearer " prefix and extra whitespace
Requirement(s)	FR-1
Type	Unit
Automated by	`JwtDecoderTest#acceptsBearerPrefixAndWhitespace`
Preconditions	Example token available.
Test Steps	1. Build the input " Bearer <token>\n". 2. Call JwtDecoder.decode() on it.
Expected Result	Token decodes. The raw token stored is the bare token with no prefix or whitespace.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-003
Title	Show header and payload as indented JSON in original key order
Requirement(s)	FR-3
Type	Unit
Automated by	`JwtDecoderTest#prettyPrintsJson`, `JsonTest#keepsKeyOrderWhenPrinting`, `JsonTest#parsesNestedValues`, `JsonTest#escapesStrings`
Preconditions	Example token available.
Test Steps	1. Decode the token and call prettyHeader() and prettyPayload(). 2. Parse and print a JSON object whose keys are not alphabetical. 3. Parse nested objects, arrays, and strings with quotes.
Expected Result	Header prints as {"alg": "HS256", "typ": "JWT"} over four indented lines. Keys keep the order they had in the token. Nested values and escaped strings survive the round trip.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-004
Title	Empty input gives a plain message
Requirement(s)	FR-4
Type	Unit
Automated by	`JwtDecoderTest#rejectsEmptyInput`
Preconditions	None.
Test Steps	1. Call decode() with " ".
Expected Result	A JwtException with the message "Paste a token to decode". No other exception.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-005
Title	Wrong number of parts gives a plain message
Requirement(s)	FR-4
Type	Unit
Automated by	`JwtDecoderTest#rejectsWrongNumberOfParts`
Preconditions	None.
Test Steps	1. Call decode() with "abc.def" (two parts).
Expected Result	A JwtException whose message says a JWT has 3 parts.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-006
Title	Bad Base64URL in the header is reported
Requirement(s)	FR-4
Type	Unit
Automated by	`JwtDecoderTest#rejectsBadBase64`
Preconditions	None.
Test Steps	1. Call decode() with "@@@.e30.sig".
Expected Result	A JwtException saying the header is not valid Base64URL.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-007
Title	Payload that is not JSON is reported
Requirement(s)	FR-4
Type	Unit
Automated by	`JwtDecoderTest#rejectsPayloadThatIsNotJson`, `JsonTest#rejectsBrokenJson`, `JsonTest#rejectsLenientJsonSyntax`
Preconditions	None.
Test Steps	1. Decode a token whose payload is Base64URL of "hello". 2. Parse {"a":}, {"a":1 (unclosed), {"a":1} extra, and [1,2] where an object is needed. 3. Parse lenient JSON: single quotes, unquoted keys, 01, trailing comma, comments.
Expected Result	Each case ends in a JwtException starting with "Invalid JSON" or saying the payload is not valid JSON. Lenient syntax is rejected, not silently accepted.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-008
Title	Claim values are shown exactly as sent
Requirement(s)	FR-3
Type	Unit
Automated by	`JsonTest#preservesLargeIntegerClaims`, `JsonTest#preservesPreciseDecimalClaims`, `JsonTest#preservesIntegersBeyondLongRange`, `JsonTest#preservesNullClaimsAndDoesNotHtmlEscapeValues`, `JwtDecoderTest#protectsNestedHeaderAndPayloadValues`, `JwtDecoderTest#snapshotsNestedValuesFromConstructorInputs`
Preconditions	None.
Test Steps	1. Parse claims 9007199254740993, 9223372036854775808, 12345.67890123456789, null values, and the text <tag>&=é. 2. Try to change the maps returned by the decoder. 3. Change the input map after building a DecodedJwt.
Expected Result	Numbers are not rounded, null stays null, and nothing is HTML escaped. The decoded maps cannot be modified and do not change when the input changes.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-009
Title	Signing algorithm is read from the header
Requirement(s)	FR-7
Type	Unit
Automated by	`JwtDecoderTest#allowsUnsignedTokenWithEmptySignature`, `JwtDecoderTest#distinguishesMissingAlgorithmFromNone`
Preconditions	None.
Test Steps	1. Decode a token with alg "none" and an empty signature. 2. Decode tokens whose header is {} and {"alg":null}.
Expected Result	alg "none" is reported as "none" with an empty signature. A missing or null alg is reported as "(missing)", not as "none".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-010
Title	Copy header, payload, and generated token
Requirement(s)	FR-8
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App running. A token decoded on the Decode screen and one generated on the Encode screen.
Test Steps	1. On Decode, click Copy next to the header, then paste into a text editor. 2. Repeat for the payload. 3. On Encode, generate a token, click Copy, paste into a text editor.
Expected Result	Each paste matches what is shown on screen exactly. One click per copy.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

5.2 Time Claims (FR-5, FR-6)

Test Case ID	TC-011
Title	Token with a future exp shows as valid
Requirement(s)	FR-6
Type	Unit
Automated by	`ClaimInspectorTest#validWhenExpInFuture`
Preconditions	ClaimInspector built with a fixed clock.
Test Steps	1. Inspect a payload whose exp is 2 hours after the clock time.
Expected Result	Status VALID with the message "Expires in 2 hours".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-012
Title	Token past exp shows as expired
Requirement(s)	FR-6
Type	Unit
Automated by	`ClaimInspectorTest#expiredWhenExpInPast`, `ClaimInspectorTest#expiredExactlyAtExp`
Preconditions	Fixed clock.
Test Steps	1. Inspect a payload whose exp is 90 seconds before the clock time. 2. Inspect a payload whose exp equals the clock time exactly.
Expected Result	Status EXPIRED. exp 90 seconds ago reads "Expired 1 minute ago". The exact boundary second also counts as expired.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-013
Title	Token before nbf shows as not valid yet
Requirement(s)	FR-6
Type	Unit
Automated by	`ClaimInspectorTest#notYetValidBeforeNbf`
Preconditions	Fixed clock.
Test Steps	1. Inspect a payload whose nbf is 30 seconds ahead and exp one hour ahead.
Expected Result	Status NOT_YET_VALID (nbf wins over a future exp), with the message "Starts in 30 seconds".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-014
Title	Token without exp shows no expiry
Requirement(s)	FR-6
Type	Unit
Automated by	`ClaimInspectorTest#noExpiryWhenExpMissing`, `ClaimInspectorTest#distinguishesNullExpiryFromMissingExpiry`
Preconditions	Fixed clock.
Test Steps	1. Inspect a payload with no exp. 2. Inspect a payload with "exp": null.
Expected Result	Missing exp gives NO_EXPIRY and "No expiry set (exp claim missing)". An exp of null is not treated as missing; it is flagged as an invalid time claim.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-015
Title	exp, iat, and nbf shown as readable local times
Requirement(s)	FR-5
Type	Unit
Automated by	`ClaimInspectorTest#formatsTimeClaimsInOrder`, `ClaimInspectorTest#formatsFractionalSecondsOnEitherSideOfTheEpoch`
Preconditions	Fixed clock in a known time zone.
Test Steps	1. Inspect a payload with iat 1516239022 and exp one hour later, clock zone UTC. 2. Inspect iat 0.5 and iat -0.5.
Expected Result	Claims are listed in the order exp, iat. iat reads "2018-01-18 01:30:22 Z". 0.5 reads 1970-01-01 00:00:00 Z and -0.5 reads 1969-12-31 23:59:59 Z (rounded down).
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-016
Title	Bad time claims are flagged, not crashed on
Requirement(s)	FR-5, FR-4
Type	Unit
Automated by	`ClaimInspectorTest#reportsNonNumericExpAsInvalid`, `ClaimInspectorTest#rejectsOutOfRangeAndNonFiniteTimestampsWithoutThrowing`, `ClaimInspectorTest#handlesDatesOutsideTheDisplayableRange`, `ClaimInspectorTest#checksAllPresentTimeClaims`
Preconditions	Fixed clock.
Test Steps	1. Inspect exp "tomorrow". 2. Inspect exp set to Long.MAX_VALUE, Long.MIN_VALUE, Double.MAX_VALUE, NaN, and both infinities. 3. Set each of exp, iat, nbf to "bad date" in turn. 4. Inspect iat equal to Instant.MAX.
Expected Result	Every case gives INVALID_TIME_CLAIM with a message starting "Invalid <claim>:" and the claim shown as "Invalid timestamp". No exception is thrown.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-017
Title	Durations read naturally
Requirement(s)	FR-6
Type	Unit
Automated by	`ClaimInspectorTest#humanDurations`
Preconditions	None.
Test Steps	1. Format 1 s, 45 min, 3 days, and -120 s.
Expected Result	Text is "1 second", "45 minutes", "3 days", and "2 minutes" (negative values use the absolute time).
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Low

5.3 Encode (FR-9 to FR-12)

Test Case ID	TC-018
Title	Encoder reproduces a known HS256 token
Requirement(s)	FR-11
Type	Unit
Automated by	`JwtEncoderTest#reproducesKnownHs256Token`
Preconditions	Known header, payload, and secret "your-256-bit-secret".
Test Steps	1. Encode the jwt.io header and payload with HS256.
Expected Result	Output is byte for byte the same as the published example token.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-019
Title	alg and typ are set from the chosen algorithm
Requirement(s)	FR-10, NFR-4
Type	Unit
Automated by	`JwtEncoderTest#setsAlgAndTypAutomatically`, `JwtEncoderTest#headerAlgIsOverriddenBySelectedAlgorithm`
Preconditions	None.
Test Steps	1. Encode with an empty header and HS384. 2. Encode with header {"alg":"none","kid":"k1"} and HS512.
Expected Result	Header gets alg HS384 and typ JWT. In the second case alg becomes HS512 (the header cannot pick its own algorithm) and kid is kept.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-020
Title	All six algorithms sign and verify
Requirement(s)	FR-10, FR-11, FR-14
Type	Unit
Automated by	`SignatureVerifierTest#roundTripsAllHmacAlgorithms`, `SignatureVerifierTest#roundTripsRsaWithPublicKey`, `JwtEncoderTest#signsWithRsaPrivateKey`
Preconditions	A fresh 2048 bit RSA key pair as PKCS#8 and SPKI PEM.
Test Steps	1. Sign a token with HS256, HS384, HS512 and verify each with the same secret. 2. Sign with RS256, RS384, RS512 and verify with the public key.
Expected Result	All six tokens are compact (three parts) and verify successfully.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-021
Title	Invalid JSON is caught before signing
Requirement(s)	FR-9
Type	Unit
Automated by	`JwtEncoderTest#rejectsInvalidPayloadJson`
Preconditions	None.
Test Steps	1. Encode with payload {"sub": }.
Expected Result	JwtException starting with "Invalid JSON". The message includes the line and column of the error. No token is produced.
Actual Result	Exception raised and no token produced (CI run #15). The test checks the "Invalid JSON" prefix only; the position text is not asserted yet (see gap G-2).
Status	**Pass**
Priority	High

Test Case ID	TC-022
Title	Missing or unusable keys are explained
Requirement(s)	FR-11
Type	Unit
Automated by	`JwtEncoderTest#rejectsMissingKey`, `JwtEncoderTest#rejectsGarbageRsaKey`, `KeyParserTest#reportsMissingAndMalformedKeys`, `KeyParserTest#rejectsNonRsaKeys`, `KeyParserTest#rejectsPublicKeyWherePrivateKeyIsRequired`
Preconditions	None.
Test Steps	1. Encode HS256 with an empty secret. 2. Encode RS256 with "not a key". 3. Parse null, empty, garbage, and a PEM with an invalid body as both key types. 4. Parse an EC key pair, and an RSA public key where a private key is required.
Expected Result	Each case gives a JwtException with a message saying what kind of key was expected, for example "Enter a secret to sign with".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-023
Title	PKCS#1 RSA private keys are accepted
Requirement(s)	FR-11
Type	Unit
Automated by	`KeyParserTest#readsPkcs1KeysAndUsesThemForRsaSigning`
Preconditions	A fresh 2048 bit RSA key pair.
Test Steps	1. Convert a fresh key pair to PKCS#1 "RSA PRIVATE KEY" and "RSA PUBLIC KEY" PEM. 2. Parse both, sign an RS256 token with the private key, verify with the public key.
Expected Result	Both keys load with the same bytes as the originals and the token verifies.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-024
Title	Numbers in claims are not rounded when signing
Requirement(s)	FR-11
Type	Unit
Automated by	`JwtEncoderTest#signsWithoutRoundingHeaderOrPayloadNumbers`
Preconditions	None.
Test Steps	1. Encode a header with 0.1234567890123456789 and a payload with 12345.67890123456789 and 9223372036854775808. 2. Decode the parts and verify.
Expected Result	Header and payload text match the input exactly and the signature verifies.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-025
Title	Open a generated token in the decoder
Requirement(s)	FR-12
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App running on the Encode screen.
Test Steps	1. Generate any HS256 token. 2. Click "Open in decoder".
Expected Result	The Decode screen opens with the new token filled in and already decoded.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

5.4 Verify (FR-13 to FR-16)

Test Case ID	TC-026
Title	Correct HMAC secret passes
Requirement(s)	FR-13
Type	Unit
Automated by	`SignatureVerifierTest#acceptsCorrectHmacSecret`
Preconditions	Example token and its secret.
Test Steps	1. Verify the example token with "your-256-bit-secret".
Expected Result	Result is valid with the message "Signature verified (HS256)".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-027
Title	Wrong HMAC secret fails with a reason
Requirement(s)	FR-13, FR-15
Type	Unit
Automated by	`SignatureVerifierTest#rejectsWrongHmacSecret`
Preconditions	Example token.
Test Steps	1. Verify with "wrong-secret".
Expected Result	Result is invalid and the message starts with "Signature does not match".
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-028
Title	Edited payload is detected
Requirement(s)	FR-13, NFR-4
Type	Unit
Automated by	`SignatureVerifierTest#detectsTamperedPayload`, `SignatureVerifierTest#verifiesOriginalBytesWithoutReformattingJson`
Preconditions	Example token.
Test Steps	1. Replace the payload with one where name is "Admin", keep the old signature, verify. 2. Sign a header and payload that contain extra spaces, newlines, and <tag>&é directly with HMAC256, then verify.
Expected Result	Tampered token fails. The untouched token with odd spacing still passes.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-029
Title	RSA token checked against a public key
Requirement(s)	FR-14
Type	Unit
Automated by	`SignatureVerifierTest#roundTripsRsaWithPublicKey`, `SignatureVerifierTest#rejectsRsaTokenWithOtherPublicKey`
Preconditions	Two different RSA key pairs.
Test Steps	1. Sign with key pair A and verify with public key A. 2. Verify the same token with public key B.
Expected Result	Public key A passes. Public key B fails.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-030
Title	alg "none" and unsupported algorithms are refused
Requirement(s)	FR-15, NFR-4
Type	Unit
Automated by	`SignatureVerifierTest#refusesAlgNone`, `SignatureVerifierTest#reportsUnsupportedAlgorithm`, `SignatureVerifierTest#reportsMissingAlgorithmInsteadOfUnsignedToken`
Preconditions	None.
Test Steps	1. Verify a token with alg "none" and the key "anything". 2. Verify a token with alg ES256. 3. Verify tokens with header {} and {"alg":null}.
Expected Result	All are invalid. Messages mention "none", read "Unsupported algorithm: ES256", and say the header has no alg value.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-031
Title	Missing key and broken token are explained
Requirement(s)	FR-15, FR-4
Type	Unit
Automated by	`SignatureVerifierTest#explainsMissingKey`, `SignatureVerifierTest#reportsMalformedTokenInsteadOfThrowing`
Preconditions	None.
Test Steps	1. Verify an HS256 token with an empty key. 2. Verify the text "not-a-token".
Expected Result	"Enter the secret that was used to sign this token", and an invalid result for the broken token. No exception reaches the caller.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	High

Test Case ID	TC-032
Title	Signature check is separate from expiry
Requirement(s)	FR-15
Type	Unit
Automated by	`SignatureVerifierTest#checksSignatureIndependentlyOfTimeClaims`, `SignatureVerifierTest#allowsInspectingMalformedDateClaimsButReportsVerificationError`
Preconditions	None.
Test Steps	1. Verify a correctly signed token with exp in 1970 and nbf, iat in 2100. 2. Sign a token whose exp is "not a date", decode it, then verify it.
Expected Result	The first token verifies (expiry is shown on the Decode screen, not by the verifier). The string exp still decodes for inspection, and verification returns an invalid result starting "Could not verify:" instead of crashing.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-033
Title	Send the decoded token to the Verify screen
Requirement(s)	FR-16
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App running. A token decoded on the Decode screen.
Test Steps	1. Click "Verify signature" on the Decode screen.
Expected Result	The Verify screen opens with the same token in its input box.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

5.5 Workspace and History (FR-17 to FR-22)

Test Case ID	TC-034
Title	Save a token with a label
Requirement(s)	FR-17
Type	Unit
Automated by	`InMemoryTokenRepositoryTest#savesAndFindsToken`, `InMemoryTokenRepositoryTest#blankLabelBecomesUntitled`, `InMemoryTokenRepositoryTest#rejectsEmptyToken`
Preconditions	Empty repository.
Test Steps	1. Save " Login token " with "abc.def.ghi ". 2. Save with a blank label. 3. Save a blank token.
Expected Result	Label and token are trimmed and listed. Blank label becomes "Untitled token". Blank token is rejected.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-035
Title	Find saved tokens by label, newest first
Requirement(s)	FR-18
Type	Unit
Automated by	`InMemoryTokenRepositoryTest#searchMatchesLabelIgnoringCase`, `FileTokenRepositoryTest#keepsSaveTimeAndOrderAfterReload`
Preconditions	Repository with "Staging admin" and "Prod reader".
Test Steps	1. Search "ADMIN". 2. Search "". 3. Save two tokens a day apart, reopen the file, list them.
Expected Result	Search is case insensitive and returns one hit; empty search returns all. List is newest first and save times are kept.
Actual Result	In memory part passed in CI run #15. File part to be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	Medium

Test Case ID	TC-036
Title	Rename and delete a saved token
Requirement(s)	FR-19
Type	Unit
Automated by	`InMemoryTokenRepositoryTest#renamesAndDeletes`
Preconditions	One saved token.
Test Steps	1. Rename to "new". 2. Rename to " ". 3. Delete it twice.
Expected Result	Rename works, blank rename returns false, first delete returns true, second returns false, list is empty.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Low

Test Case ID	TC-037
Title	Sample tokens decode and verify
Requirement(s)	FR-17
Type	Integration
Automated by	`InMemoryTokenRepositoryTest#sampleDataDecodesAndVerifiesWithDemoSecret`
Preconditions	Repository built with sample data.
Test Steps	1. Decode each of the 3 sample tokens. 2. Verify each with "jwtoffline-demo-secret".
Expected Result	Three tokens, all decode, all verify. Checks repository, decoder, and verifier together.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Low

Test Case ID	TC-038
Title	Workspace table and opening a saved token
Requirement(s)	FR-18, FR-20
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App running with the sample tokens in the workspace.
Test Steps	1. Open the Workspace screen and check the columns. 2. Type "pay" in the filter box. 3. Double click the remaining row. 4. Go back, select a row, click "Open in decoder".
Expected Result	Columns show label, algorithm, status, and save time. Filter leaves only "Payments service (HS512)". Both double click and the button open the token on the Decode screen.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

Test Case ID	TC-039
Title	Recent history on the Home screen
Requirement(s)	FR-21
Type	Unit
Automated by	`TokenHistoryTest#newestFirstWithoutDuplicates`, `TokenHistoryTest#deduplicatesBearerAndRawTokens`, `TokenHistoryTest#dropsOldestPastLimit`, `TokenHistoryTest#ignoresBlankTokens`, `TokenHistoryTest#limitMustBePositive`
Preconditions	Empty TokenHistory.
Test Steps	1. Add a, b, a. 2. Add " Bearer a.b.c", "a.b.c", and " bEaReR a.b.c ". 3. With limit 2, add 1, 2, 3. 4. Add a blank token and null. 5. Create a history with limit 0.
Expected Result	Order is a, b with no duplicate. All Bearer forms count as one token. With limit 2 the list is 3, 2. Blank and null are ignored. Limit 0 is rejected. The app uses the default limit of 20.
Actual Result	Matched expected result. All assertions passed in CI run #15 (development, Oct 11, 2026).
Status	**Pass**
Priority	Low

Test Case ID	TC-040
Title	Workspace survives closing and reopening
Requirement(s)	FR-22
Type	Unit
Automated by	`FileTokenRepositoryTest#savedTokensSurviveReopening`, `FileTokenRepositoryTest#renameAndDeleteArePersisted`, `FileTokenRepositoryTest#searchWorksAfterReload`
Preconditions	Temporary folder from JUnit @TempDir.
Test Steps	1. Save a token, then open a new FileTokenRepository on the same file. 2. Save two, rename one, delete the other, reopen. 3. Save two, reopen, search.
Expected Result	Reopened repository has the same tokens with the same ids, labels, and times. Rename and delete are kept. Search works on reloaded data.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-041
Title	First launch creates the workspace file
Requirement(s)	FR-22
Type	Unit
Automated by	`FileTokenRepositoryTest#firstLaunchCreatesFileWithSampleTokens`, `FileTokenRepositoryTest#missingFileWithoutSeedingStartsEmpty`
Preconditions	File and its folder do not exist.
Test Steps	1. Open with sample seeding on. 2. Open with sample seeding off.
Expected Result	With seeding, the folder and file are created and hold the 3 sample tokens. Without seeding, the workspace is empty and no file is written until the first save.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	Medium

Test Case ID	TC-042
Title	Damaged workspace file does not stop the app
Requirement(s)	FR-22, NFR-5
Type	Unit
Automated by	`FileTokenRepositoryTest#corruptFileStartsEmptyAndKeepsBackup`, `FileTokenRepositoryTest#entryWithMissingFieldsIsTreatedAsCorrupt`
Preconditions	workspace.json contains "{ this is not json" or an entry with no token.
Test Steps	1. Open the repository on the damaged file.
Expected Result	Workspace starts empty, a warning is available, and the old file is copied to workspace.json.corrupt so nothing is lost.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-043
Title	Saved tokens are there after restarting the app
Requirement(s)	FR-22
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App installed. Delete ~/.jwtoffline/workspace.json before starting.
Test Steps	1. Start the app and open Workspace (3 sample tokens). 2. Decode any token, save it as "Restart test". 3. Close the app fully and start it again. 4. Open Workspace.
Expected Result	"Restart test" is listed with the 3 samples. Renames and deletes made before closing are also kept.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	High

5.6 Navigation (FR-23)

Test Case ID	TC-044
Title	Side navigation keeps each screen's input
Requirement(s)	FR-23
Type	Manual UI
Automated by	Manual (no automated test)
Preconditions	App running.
Test Steps	1. Paste a token on Decode. 2. Switch to Encode, type a payload. 3. Switch to Verify, then back to Decode and Encode.
Expected Result	One window. Every screen is reachable from the side menu, and the token and payload typed earlier are still there.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	High

5.7 Non-Functional Requirements

Test Case ID	TC-045
Title	No networking code in the app
Requirement(s)	NFR-1
Type	Static
Automated by	`ArchitectureTest#noSourceFileUsesNetworking`, `ArchitectureTest#sourcesAreFound`
Preconditions	Tests run from the project folder.
Test Steps	1. Scan every .java file under src/main/java for java.net, HttpURLConnection, and Socket usage.
Expected Result	No file matches. The guard test confirms the scan actually found the source files.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-046
Title	Secrets are never written to disk
Requirement(s)	NFR-1
Type	Unit
Automated by	`FileTokenRepositoryTest#fileStoresOnlyTokensAndLabels`
Preconditions	Temporary folder.
Test Steps	1. Save a token signed with the demo secret. 2. Read workspace.json as text.
Expected Result	The file holds the label and token but not the secret.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-047
Title	App works with the network turned off
Requirement(s)	NFR-1
Type	Manual
Automated by	Manual (no automated test)
Preconditions	Wi-Fi off and cable unplugged.
Test Steps	1. Start the app. 2. Decode, encode, verify, and save a token.
Expected Result	Every feature works with no error or delay.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

Test Case ID	TC-048
Title	Decode an 8 KB token in under 100 ms
Requirement(s)	NFR-2
Type	Performance
Automated by	`PerformanceTest#decodes8kbTokenUnder100ms`
Preconditions	An HS256 token of about 8 KB built in @BeforeAll.
Test Steps	1. Warm up with 5 decodes. 2. Time 21 decodes and take the median.
Expected Result	Median is under 100 ms.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-049
Title	Verify an 8 KB token in under 100 ms
Requirement(s)	NFR-2
Type	Performance
Automated by	`PerformanceTest#verifiesHmac8kbTokenUnder100ms`, `PerformanceTest#verifiesRsa8kbTokenUnder100ms`
Preconditions	8 KB HS256 and RS256 tokens and their keys.
Test Steps	1. Warm up 5 times. 2. Time 21 verifications of each and take the median.
Expected Result	Both verify as valid and both medians are under 100 ms.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-050
Title	App starts in under 3 seconds
Requirement(s)	NFR-2
Type	Manual
Automated by	Manual (no automated test)
Preconditions	Typical student laptop, app built with mvn package.
Test Steps	1. Start a stopwatch and launch the app. 2. Stop when the Home screen is usable. Repeat 3 times.
Expected Result	Every start is under 3 seconds.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

Test Case ID	TC-051
Title	New user decodes a token within 30 seconds
Requirement(s)	NFR-3
Type	Usability
Automated by	Manual (no automated test)
Preconditions	Three classmates who have not used the app. A sample token on the clipboard.
Test Steps	1. Ask each person to "find out who this token belongs to" with no instructions. 2. Time until they read the sub claim. Note anything confusing.
Expected Result	All three succeed in 30 seconds or less.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

Test Case ID	TC-052
Title	Random input never crashes the logic
Requirement(s)	NFR-5, FR-4
Type	Robustness
Automated by	`RobustnessTest#decoderOnlyThrowsJwtExceptionForRandomInput`, `RobustnessTest#encoderOnlyThrowsJwtExceptionForRandomJson`, `RobustnessTest#verifierNeverThrowsAndNeverPassesGarbage`
Preconditions	Seeded random generators so every run uses the same inputs.
Test Steps	1. Send 2,000 random strings and random three part tokens to the decoder. 2. Send 1,000 random header and payload pairs to the encoder. 3. Send 1,000 random tokens and keys to the verifier.
Expected Result	Decoder and encoder only ever throw JwtException, which every screen catches and shows. Verifier never throws and never reports a random token as valid.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-053
Title	Number with a huge exponent (regression for D-01)
Requirement(s)	NFR-5, FR-4
Type	Robustness
Automated by	`RobustnessTest#hugeExponentIsReportedAsInvalidJson`
Preconditions	None.
Test Steps	1. Parse {"exp":1e9999999999}. 2. Decode and verify a token whose payload is that JSON.
Expected Result	A JwtException with a readable message. Before the fix this threw NumberFormatException, which the screens do not catch.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	High

Test Case ID	TC-054
Title	Build and tests pass on Linux with JDK 17
Requirement(s)	NFR-6
Type	CI
Automated by	`.github/workflows/build.yml`
Preconditions	Push to a tracked branch.
Test Steps	1. GitHub Actions runs mvn -B verify on ubuntu-latest with Temurin 17.
Expected Result	Build succeeds and the JaCoCo report is uploaded.
Actual Result	Runs #1 to #15 all succeeded (Oct 10 to 11, 2026).
Status	**Pass**
Priority	Medium

Test Case ID	TC-055
Title	App runs on Windows and macOS
Requirement(s)	NFR-6
Type	Manual
Automated by	Manual (no automated test)
Preconditions	One Windows and one macOS machine with JDK 17 or newer.
Test Steps	1. Clone the repo and run mvn javafx:run. 2. Decode, encode, verify, and save a token.
Expected Result	App starts and every screen works on both systems.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Medium

Test Case ID	TC-056
Title	Logic packages have no UI code
Requirement(s)	NFR-7
Type	Static
Automated by	`ArchitectureTest#corePackageHasNoUiCode`, `ArchitectureTest#workspacePackageHasNoUiCode`
Preconditions	Tests run from the project folder.
Test Steps	1. Scan core and workspace sources for javafx imports.
Expected Result	No file in core or workspace imports JavaFX.
Actual Result	To be filled after the ITR2.1 CI run.
Status	**Pending**
Priority	Medium

Test Case ID	TC-057
Title	Line coverage of core and workspace is at least 80%
Requirement(s)	NFR-7
Type	Coverage
Automated by	`JaCoCo (mvn verify)`
Preconditions	Full test suite runs.
Test Steps	1. Run mvn verify. 2. Open target/site/jacoco/index.html and read line coverage for the core and workspace packages.
Expected Result	Both packages are at 80% line coverage or higher.
Actual Result	To be read from the JaCoCo report attached to the ITR2.1 CI run. Full coverage analysis is planned for Deliverable 3.1.
Status	**Pending**
Priority	Medium

Test Case ID	TC-058
Title	Keyboard only use and readable status
Requirement(s)	NFR-8
Type	Manual
Automated by	Manual (no automated test)
Preconditions	App running. Mouse unplugged.
Test Steps	1. Use Tab, Shift+Tab, Enter, and Space to decode, encode, verify, and save. 2. Check text contrast of banners with a contrast checker.
Expected Result	Every action is reachable by keyboard. Text meets WCAG AA. Status banners use words, not only color.
Actual Result	Not run yet. Scheduled for the Week 10 manual test session (before Deliverable 2.2).
Status	**Not run**
Priority	Low

6. Test Results

The spec allows actual results and status to be reported in later iterations. This section records where every test case stands at submission.

6.1 Summary

Status	Test cases	Notes
Pass	35	Automated cases whose tests ran in CI run #15 on the development branch (commit c6c9db3, Oct 11, 2026): 70 tests, 0 failures. Includes the CI build check for NFR-6.
Pending	12	Automated cases with tests written this iteration (20 new tests). They run for the first time in CI when ITR2.1 is pushed.
Not run	11	Manual cases. Scheduled for the manual test session in Week 10, before Deliverable 2.2.
Fail	0	No open failures. D-01 below was found and fixed in this iteration.

6.2 Automated Test Inventory

Test class	Tests	New in ITR2.1	Requirements
JwtDecoderTest	11		FR-1 to FR-4, FR-7
JsonTest	9		FR-3, FR-4, FR-9
ClaimInspectorTest	13		FR-4 to FR-6
JwtEncoderTest	8		FR-9 to FR-11, NFR-4
KeyParserTest	4		FR-11, FR-14
SignatureVerifierTest	14		FR-10, FR-13 to FR-15, NFR-4
InMemoryTokenRepositoryTest	6		FR-17 to FR-19
TokenHistoryTest	5		FR-21
FileTokenRepositoryTest	9	Yes	FR-18, FR-22, NFR-1, NFR-5
PerformanceTest	3	Yes	NFR-2
RobustnessTest	4	Yes	FR-4, NFR-5
ArchitectureTest	4	Yes	NFR-1, NFR-7
Total	90	20	

6.3 Defect Log

ID	Found by	Description	Fix	Status
D-01	Writing TC-053 (NFR-5)	A JSON number with a huge exponent, for example {"exp":1e9999999999}, made BigDecimal throw NumberFormatException. Gson does not wrap that exception, so it escaped Json.parse(). The screens only catch JwtException, so pasting such a token would have left the Decode screen without an error message. This broke FR-4 and NFR-5.	Json.parse() now also catches NumberFormatException and reports "Invalid JSON: a number is too large to read". Regression test RobustnessTest#hugeExponentIsReportedAsInvalidJson.	Fixed, waiting for CI to confirm

7. Prototype Overview

7.1 What Changed Since ITR1.2

Change	Requirement	Details
Workspace saved to a file	FR-22, US-4.5	New FileTokenRepository writes the workspace to ~/.jwtoffline/workspace.json after every save, rename, or delete, and loads it on startup. Writes go to a temp file first and are then moved into place, so a crash mid write cannot leave half a file. On first launch it adds the three sample tokens. A damaged file is copied to workspace.json.corrupt and the app starts with an empty workspace instead of failing.
Shared repository logic	NFR-7	InMemoryTokenRepository now has a changed() hook. FileTokenRepository extends it and only adds loading and saving, so search, rename, and delete are written once. The screens still use the TokenRepository interface and did not change; only AppContext picks the new class.
Crash fix D-01	FR-4, NFR-5	See Section 6.3.
Library refactor (PR #2, Steven)	FR-11, NFR-6	Merged on Oct 11 after ITR1.2. Replaced our own JSON parser with Gson in strict mode, signing and verifying with Auth0 java-jwt, and PEM parsing with Bouncy Castle, which also added PKCS#1 key support. All 70 tests passed after the change.
Test traceability in code	All	Every test class header now names its user stories, requirements, and test case IDs. Several headers from ITR1.2 pointed at old requirement numbers and were corrected.

7.2 Requirement Updates

Two requirements were reworded to match the code after the library refactor. The rest are unchanged from Deliverable 1.2.

ID	Deliverable 1.2 wording	Updated wording	Reason
FR-11	Sign with an HMAC secret or a PKCS#8 PEM RSA private key.	Sign with an HMAC secret or an unencrypted PKCS#8 or PKCS#1 PEM RSA private key.	PKCS#1 support (backlog item 8) arrived early with Bouncy Castle.
NFR-6	No native code and no runtime libraries beyond the JDK and JavaFX.	No native code. Runtime libraries are limited to pure Java libraries that make no network calls (currently Gson, Auth0 java-jwt, Bouncy Castle).	Using well tested libraries for crypto and JSON is safer than our own code. NFR-1 (no network) still holds and TC-045 checks it.

7.3 Running the Prototype

Install JDK 17 or newer and Maven, clone the repository, then run `mvn javafx:run`. The Workspace screen opens with three sample tokens signed with `jwtoffline-demo-secret`. Saved tokens are now kept between runs. To start fresh, delete ~/.jwtoffline/workspace.json.

8. Code Repository and Branching Strategy

Repository: https://github.com/isad07/JWToffline

Branch	Purpose	State at ITR2.1
main	Stable code. Updated from development at each deliverable.	Tagged ITR1.2 and ITR2.1
development	Integration branch. Feature branches merge here through pull requests after CI passes and a review.	Active
feature/prototype-itr1.2	ITR1.2 prototype screens and first tests.	Merged (PR #1)
refactor/app-cleanup	Move to Gson, java-jwt, and Bouncy Castle.	Merged (PR #2)
feature/testing-itr2.1	File workspace, D-01 fix, 20 new tests, this document.	Merged for ITR2.1

Every push to these branches runs the CI build. The tag `ITR2.1` marks this submission, following the `ITR1.2` tag from the last deliverable.

9. Task Allocation and Timeline

9.1 Task Breakdown

Task	Description	Requirements	Owner	Est. (h)	Status
T18	FileTokenRepository and load on startup	FR-22	Steven	6	Done
T19	FileTokenRepositoryTest (9 tests)	FR-22, NFR-1	Steven	3	Done
T20	Test plan, traceability matrix, test case write up	All	Isa	8	Done
T21	RobustnessTest and fix for D-01	FR-4, NFR-5	Isa	3	Done
T22	ArchitectureTest (no network, no UI in logic)	NFR-1, NFR-7	Isa	1	Done
T23	PerformanceTest for 8 KB tokens	NFR-2	Osmond	2	Done
T24	Library refactor review and test updates	FR-11, NFR-6	Steven	4	Done
T25	Fix requirement IDs in test class headers	All	Osmond	1	Done
T26	Manual UI test session (TC-010, 025, 033, 038, 043, 044)	FR-8, 12, 16, 18, 20, 22, 23	Saeed, Osmond	3	Week 10
T27	Platform, offline, startup checks (TC-047, 050, 055)	NFR-1, NFR-2, NFR-6	Saeed	2	Week 10
T28	Hallway usability and keyboard checks (TC-051, 058)	NFR-3, NFR-8	Saeed	2	Week 10
T29	Close gaps G-1, G-2, G-4	FR-7, FR-9, FR-22	Isa	3	Iteration 2.2

9.2 Sprint Plan (Weeks 6 to 9)

Week	Plan
Week 6 to 7	Implement core features and set up unit tests: FileTokenRepository and its tests (T18, T19). Merge the library refactor and update tests (T24). Draft the test plan.
Week 8	Refine features and run tests: performance, robustness, and architecture tests (T21 to T23). Fix D-01. Build the traceability matrix and look for gaps.
Week 9	Finalize the prototype and gather feedback: write up all test cases, fix test headers (T25), tag ITR2.1, submit on November 8.

10. Updated Iteration Plan

Iteration	Due	Focus
1.1 Vision	Sep 20	Done.
1.2 Requirements and prototype	Oct 11	Done.
2.1 Testing (this document)	Nov 8	Test plan, traceability, 90 automated tests, file workspace.
2.2 Full documentation, prototype, presentation	Nov 29	Run all manual test cases and fill in actual results. Close gaps G-1, G-2, G-4. Peer review. User manual. Syntax highlighting and keyboard shortcuts.
3.1 Design documentation	Jan 31	Class, sequence, and activity diagrams. TestFX GUI tests and JaCoCo coverage analysis (closes G-3).
3.2 Completed implementation	Mar 7	Refactoring, ES256 support, JWKS import, integration and edge case testing.
3.3 Final submission	Mar 28	Final SRS, SDD, testing document, user manual, presentation and demo.

Updated Backlog

Rank	Item	Type	Priority	Target
1	Run manual test cases and record results (T26 to T28)	Test	High	Iteration 2.2
2	Show workspace load warning on screen (G-4)	Story	High	Iteration 2.2
3	Signature length helper and test (G-1), JSON error position assertion (G-2)	Test	Medium	Iteration 2.2
4	Syntax highlighting for header and payload JSON	Story	Medium	Iteration 2.2
5	Keyboard shortcuts and full keyboard navigation (NFR-8)	Story	Medium	Iteration 2.2
6	TestFX GUI tests for all five screens (G-3)	Test	Medium	Iteration 3.1
7	ES256 and ES384 signing and verification	Story	Medium	Iteration 3.2
8	Import a public key from a JWKS file	Story	Low	Iteration 3.2
9	Export and import the workspace	Story	Low	Iteration 3.2
10	Optional password protection for the workspace file	Story	Low	Stretch

Accepting PKCS#1 keys (rank 8 in Deliverable 1.2) and saving the workspace to a file (rank 1) are now done and have been removed from the backlog.